

```

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score
from sklearn.linear_model import LogisticRegression

```

```

# 1-1 Upload first data set
hr1_df = pd.read_csv('HRData1.csv')
hr1_shape = hr1_df.shape
hr1_data_rows, hr1_data_cols = hr1_shape[0], hr1_shape[1]

# 1-2 Upload second data set
hr2_df = pd.read_csv('HRData2.csv')
hr2_shape = hr2_df.shape
hr2_data_rows, hr2_data_cols = hr2_shape[0], hr2_shape[1]

# 1-3 Merge the two data sets
hr_merged_df = pd.merge(hr1_df, hr2_df, how='inner', on='EmployeeNumber')
hr_merged_shape = hr_merged_df.shape
hr_merged_data_rows, hr_merged_data_cols = hr_merged_shape[0],
hr_merged_shape[1]

print(f'HR 1 data:  rows={hr1_data_rows}, cols={hr1_data_cols}')
print(f'HR 2 data:  rows={hr2_data_rows}, cols={hr2_data_cols}')
print(f'Merged data: rows={hr_merged_data_rows},
cols={hr_merged_data_cols}')

```

HR 1 data: rows=1470, cols=10
 HR 2 data: rows=1470, cols=21
 Merged data: rows=1470, cols=30

```

# 2-1 Create High Income feature
hr_merged_df['high_income'] = 0
hr_merged_df.loc[hr_merged_df['MonthlyIncome'] >= 8000, 'high_income'] = 1

# 2-2 Subset Data with features high_income, NumCompaniesWorked and target
variable Attrition

```

```
data_model_df = hr_merged_df[['high_income', 'NumCompaniesWorked',  
'Attrition']].copy()  
data_model_shape = data_model_df.shape  
  
print(f"Shape:    rows={data_model_shape[0]}, cols={data_model_shape[1]}")  
Shape: rows=1470, cols=3
```

```
# 3-1 Split data into Feature and Target Vectors  
target = 'Attrition'  
X = data_model_df.drop(target, axis=1)  
y = data_model_df[target]  
  
# 3-2 Create Training and Testing Datasets  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=55)  
  
print(f"Training set:  X={X_train.shape}, y={y_train.shape}")  
print(f"Test set:      X={X_test.shape}, y={y_test.shape}")  
Training set: X=(1176, 2), y=(1176,)  
Test set:     X=(294, 2), y=(294,)
```

```
# 4-1 Train the Model  
clf = LogisticRegression(random_state=55).fit(X_train, y_train)  
  
# 4-2 Make Inference on Train and Test Sets  
y_train_pred = clf.predict(X_train)  
y_test_pred = clf.predict(X_test)  
  
# 4-3 Report Metrics  
# accuracy_score, precision_score, recall_score  
train_accuracy = accuracy_score(y_train, y_train_pred)  
train_precision = precision_score(y_train, y_train_pred)  
train_recall = recall_score(y_train, y_train_pred)  
print(f"\ntrain set metrics")  
print(f"accuracy = {round(train_accuracy, 2)}")  
print(f"precision = {round(train_precision, 2)}")  
print(f"recall = {round(train_recall, 2)}")  
  
test_accuracy = accuracy_score(y_test, y_test_pred)  
test_precision = precision_score(y_test, y_test_pred)  
test_recall = recall_score(y_test, y_test_pred)
```

```
print(f"\ntest set metrics")
print(f"accuracy = {round(test_accuracy, 2)}")
print(f"precision = {round(test_precision, 2)}")
print(f"recall = {round(test_recall, 2)}")
```

train set metrics

accuracy = 0.73

precision = 0.54

recall = 0.45

test set metrics

accuracy = 0.8

precision = 0.55

recall = 0.56

```
# 5-1 Merge predictions with original data

## Merge predictions with the original train data
train_pred_df = pd.DataFrame({'Attrition_Pred': y_train_pred},
                              index=X_train.index)
train_actual_df = pd.DataFrame({'Attrition': y_train},
                                index=X_train.index)
train_merged_df = pd.concat([X_train, train_actual_df, train_pred_df],
                             axis=1)

## Merge predictions with the original test data
test_pred_df = pd.DataFrame({'Attrition_Pred': y_test_pred},
                              index=X_test.index)
test_actual_df = pd.DataFrame({'Attrition': y_test}, index=X_test.index)
test_merged_df = pd.concat([X_test, test_actual_df, test_pred_df], axis=1)
```

```
# 6-1 Subset Training and Testing set to Only Include High Income Category
train_high_inc_df =
train_merged_df[train_merged_df['high_income']==1].copy()
test_high_inc_df = test_merged_df[test_merged_df['high_income']==1].copy()

# 6-2 Get Actual and Predicted Attrition Data
y_train = train_high_inc_df['Attrition'].values
y_train_pred = train_high_inc_df['Attrition_Pred'].values

y_test = test_high_inc_df['Attrition'].values
```

```

y_test_pred = test_high_inc_df['Attrition_Pred'].values
# 6-3 Report Metrics
# report metrics accuracy_score, precision_score, recall_score
train_accuracy = accuracy_score(y_train, y_train_pred)
train_precision = precision_score(y_train, y_train_pred)
train_recall = recall_score(y_train, y_train_pred)
print(f"\ntrain set metrics")
print(f"accuracy = {round(train_accuracy, 2)}")
print(f"precision = {round(train_precision, 2)}")
print(f"recall = {round(train_recall, 2)}")

test_accuracy = accuracy_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred)
test_recall = recall_score(y_test, y_test_pred)
print(f"\ntest set metrics")
print(f"accuracy = {round(test_accuracy, 2)}")
print(f"precision = {round(test_precision, 2)}")
print(f"recall = {round(test_recall, 2)}")

```

train set metrics

accuracy = 0.53

precision = 0.54

recall = 0.9

test set metrics

accuracy = 0.61

precision = 0.55

recall = 0.97